

UTF-8 Validation with Processor-Independent SIMD

Design, correctness, and evaluation plan for two Parabix implementations

[Team member names] | CMPT 980 | August 5, 2026

Research question How effectively can Parabix/IDISA abstractions express UTF-8 validation, and how does compiler-generated SIMD compare with the hand-written Keiser-Lemire approach?

1. Introduction

UTF-8 validation is a small algorithm with unusually demanding boundary behavior. A validator must recognize ASCII and multibyte sequences, reject overlong encodings and surrogate code points, enforce the U+10FFFF limit, and carry sequence state across vector and file boundaries. The project studies whether those rules can be stated using processor-independent Parabix concepts while retaining the performance benefits of SIMD compilation.

The work contributes two Parabix validators. The direct implementation operates on packed byte vectors through IDISA comparisons, shifts, masks, and population counts. The Pablo implementation strips the validation logic from the existing u8u16 transcoder, retaining its scope and error expressions while removing all UTF-16 production, deletion, compression, and output stages. The study compares these compiler-generated implementations with a hand-written Keiser-Lemire implementation and a scalar reference.

The correctness investigation was motivated by a specific streaming question: whether a file ending in an incomplete sequence is rejected when its length is exactly a multiple of the SIMD block size. The original implementation did not reject that case. The completed implementation makes EOF an explicit part of validation and verifies it systematically.

2. Background

The Unicode Standard defines the permitted byte ranges for well-formed UTF-8. Continuation bytes occupy 80-BF; legal two-byte leaders begin at C2; E0, ED, F0, and F4 require narrower second-byte ranges; and no encoded value may represent a surrogate or exceed U+10FFFF [1].

Keiser and Lemire describe a lookup-oriented SIMD validator designed to use fewer than one instruction per input byte on supported machines [2]. It provides the hand-written comparison point for this compiler study. Parabix instead expresses parallel text processing through target-independent stream and IDISA operations that are lowered through LLVM. The repository's u8u16 transcoder provides a mature Pablo expression of UTF-8 scopes, but its original purpose is UTF-16 generation rather than validation alone [3].

3. Design

3.1 Shared validation model

Both implementations compute where continuation bytes are required. A two-byte leader creates one required position; a three-byte leader creates two; and a four-byte leader creates three. XOR between the required-position mask and the actual 80-BF mask identifies missing or unexpected continuations. Additional range predicates reject C0/C1, F5-FF, E0 80-9F, ED A0-BF, F0 80-8F, and F4 90-BF.

Implementation	Representation	Primary operations	EOF strategy
Scalar reference	Bytes	Branching state machine	Decoder rejects unfinished state
Direct IDISA	Packed 8-bit lanes	Vector comparisons, dslli, XOR, popcount	Add1 EOF guard plus block state
Pablo	Eight basis bit streams	Character classes, Advance, boolean scopes	AtEOF(anyScope)
Keiser-Lemire	Native SIMD vectors	Lookup/shuffle-oriented intrinsics	Library-specific tail path

3.2 Direct byte-oriented kernel

The direct kernel is block-oriented. It retains the preceding bit block in a persistent pending scalar and uses `mvmd_dslli` to align bytes one, two, and three positions behind each current byte. Unsigned byte comparisons create the continuation, leader, illegal-byte, and narrow-range masks. `hsimd_signmask` and `popcount` reduce error lanes to the scalar error count.

The earlier MultiBlock implementation detected most truncated files because a partial final block was zero padded, but it missed truncation at exact block multiples. The refactor gives the byte kernel explicit final-block behavior and adds a small direct-byte Pablo EOF guard. The guard advances only legal leaders, applies `AtEOF` to the outstanding scope, and contributes one error at the Add1 position. This guarantees that the correctness result does not depend on whether the source happens to schedule a padded block.

3.3 Validation-only Pablo kernel

The second implementation starts from the logical validation portion of `u8u16`. It classifies leaders and continuations from the eight S2P basis streams, forms `scope22`, `scope32/scope33`, and `scope42/scope43/scope44` with `Advance`, and emits only an error stream. No UTF-16 planes, selector masks, deletion, compression, P2S, or sink remain. A custom counter includes the Add1 EOF position, unlike a conventional in-file-only `popcount`.

4. Implementation Details

- The command line preserves `utf8validate` and adds `--algorithm=direct|pablo`; `direct` is the default.
- Both paths report `VALID` only when their error count is zero and return a nonzero process status for invalid input.
- The Pablo path performs `Selected_S2P`, validation-only scope construction, and error reduction.
- The direct path performs packed-byte validation and a narrow EOF-only guard on the original byte stream.

- The generated suite invokes both algorithms at 64, 128, 256, and 512-bit block sizes and compares them with Python strict UTF-8 decoding.

Reproduction Build with ``make utf8validate``, then run ``python3 Update2Work/tests/run_utf8_validation_tests.py``. A focused run accepts ``--block-bits 256``.

5. Implementation Evaluation

5.1 Correctness

The deterministic generator produced 124 cases per block width and 496 cases overall. Cases include empty and one-byte files, ASCII and multilingual text, stray continuations, illegal leaders, overlong sequences, surrogates, values above U+10FFFF, valid sequences crossing boundaries, and truncated two-, three-, and four-byte sequences around and exactly at one, two, and three block boundaries.

Oracle / implementation	64	128	256	512	Overall
Python strict UTF-8	124/124	124/124	124/124	124/124	496/496
Direct IDISA + EOF guard	124/124	124/124	124/124	124/124	496/496
Pablo validation-only	124/124	124/124	124/124	124/124	496/496

The original direct implementation was reproduced as accepting incomplete sequences at exact block multiples. After the EOF changes, both Parabix implementations agreed with the oracle on every generated case.

5.2 Performance methodology and pending results

Groupmate action required Replace this callout with verified benchmark charts and raw-data references before submission. Do not infer or invent throughput values.

The benchmark contract requires the same machine, compiler settings, corpora, and validity semantics across implementations. Compilation/JIT, file I/O, and steady-state validation must be reported separately. Each configuration receives warmup runs and at least ten measured trials; the report uses median GB/s and interquartile range. Single-thread performance precedes 1/2/4/8-thread scaling, capped at physical cores.

Environment item	Recorded value
CPU	AMD Ryzen 7 7700X, 8 physical cores / 16 logical CPUs
SIMD features	AVX2 and AVX-512 available under WSL
C++ compiler	GCC 13.3.0; Release build uses -O3
LLVM	16.0.6
Repository baseline	3dcd1cb; final experiment commit: [insert hash]
Raw results	[insert CSV path and benchmark script commit]

[Insert the groupmate's single-thread chart, multithread scaling chart, and interpretation here. The interpretation must distinguish algorithmic work from S2P conversion, JIT/setup, I/O, and synchronization overhead.]

6. Conclusion: Lessons Learned and Further Work

The project demonstrates that UTF-8 scope rules can be stated compactly in both byte-oriented IDISA and basis-bit Pablo forms. The strongest correctness lesson is that vector logic alone is insufficient for a streaming validator: EOF is a semantic position, and exact block boundaries must be tested independently of partial-block padding. The Pablo AtEOF construct made that condition explicit, while the direct path required a deliberate EOF guard.

The performance conclusion remains contingent on the reproducible benchmark results. The evaluation should determine whether the lower-level direct representation offsets the additional EOF kernel and whether the Pablo representation's S2P cost is recovered through simpler boolean logic. Further work should inspect generated LLVM and assembly, isolate S2P and EOF-guard costs, test on ARM/NEON, and evaluate chunk-parallel validation with explicit boundary handoff.

References

- [1] Unicode Consortium. The Unicode Standard, Chapter 3, Conformance, Table 3-7.
<https://www.unicode.org/versions/latest/ch03.pdf>
- [2] J. Keiser and D. Lemire. Validating UTF-8 In Less Than One Instruction Per Byte. *Software: Practice and Experience* 51(5), 2021. <https://arxiv.org/abs/2010.03090>
- [3] Parabix repository and u8u16 implementation used in this project. <https://github.com/Grendlee/GrantPrit>
- [4] simdutf project. Unicode routines using SSE2, AVX2, NEON, AVX-512 and other targets.
<https://github.com/simdutf/simdutf>

Appendix A. Source and Script Map

Artifact	Purpose
Update2Work/utf8_validation_kernel.*	Direct packed-byte block kernel
Update2Work/utf8_pablo_validation_kernel.*	Pablo validator, EOF guard, and Add1-aware counter
Update2Work/utf8validate.cpp	CLI selection, pipeline construction, reporting
Update2Work/tests/run_utf8_validation_tests.py	Generated differential and boundary tests
Update2Work/BENCHMARK_HANDOFF.md	Benchmark contract and expected outputs

Appendix B. Contribution Record

[Student name]: EOF investigation and fix, validation-only Pablo implementation, correctness generator, design/implementation writing, conclusion draft, poster technical slides, benchmark review.

[Groupmate name]: Keiser-Lemire integration, benchmark automation, raw measurements, plots, evaluation draft, performance presentation.